

It does this using the following conditional (which is not meant to be run alone):

```
if((distance2 > distance1 and randomdraw < temperature) or (distance1 > distance2)):
```

We may want to change that condition so that our willingness to accept a worse itinerary depends not only on the temperature but also on how much worse our hypothetical change makes the itinerary. If it makes it just a little worse, we'll be more willing to accept it than if it makes it much worse. To account for this, we'll incorporate into our conditional a measurement of how much worse our new itinerary is. The following conditional (which is also not meant to be run alone) is an effective way to accomplish this:

```
scale = 3.5
if((distance2 > distance1 and (randomdraw) < (math.exp(scale*(distance1-
distance2)) * temperature)) or (distance1 > distance2)):
```

When we put this conditional in our code, we have the function in [Listing 6-6](#), where we show only the very end of the `perturb()` function.

```
--snip--
# beginning of perturb function goes here

    scale = 3.5
    if((distance2 > distance1 and (randomdraw) < (math.exp(scale * (distance1 -
distance2)) * temperature)) or (distance1 > distance2)):
        itinerarytoreturn = itinerary2.copy()

    return(itinerarytoreturn.copy())
```

Allowing Resets

During the simulated annealing process, we may unwittingly accept a change to our itinerary that's unequivocally bad. In that case, it may be useful to keep track of the best itinerary we've encountered so far and allow our algorithm to reset to that best itinerary under certain conditions. [Listing 6-6](#) provides the code to do this, highlighted in bold in a new, full perturbing function for simulated annealing.